

PLATFORM DOCUMENTATION

7ALP's

A complete walkthrough of the storefront, admin console, and B2B wholesale portal — every login, route, and feature, as currently implemented.

Updated 2026-10-06 · Frontend: React 19 + Vite + Redux Toolkit + Tailwind 4 ·
Backend: Node.js + Express 5 + MongoDB (Mongoose)

1. Architecture Overview

The platform is a two-app monorepo: a React single-page app (`frontend/`) talking to a REST API (`backend/`) mounted under `/api/v1` . There are three independent authentication systems — Customer, Admin, and B2B — each with its own JWT stored in an httpOnly cookie, so a login in one role never grants access to another.

Frontend stack

- React 19, React Router 7
- Redux Toolkit (cart, auth, wishlist, B2B draft-quote state)
- Tailwind CSS 4
- Axios (shared instance, cookie credentials on)
- Google Maps JS API (Places + Geocoding)
- Razorpay Checkout.js

Backend stack

- Express 5, MongoDB via Mongoose
- JWT + httpOnly cookies per role
- Cloudflare R2 for product images
- Razorpay Node SDK
- Resend for transactional email (OTPs / password reset)
- Helmet, rate limiting, custom sanitize middleware

2. Logins

Customer CHANGED

Email + OTP, no password — switched this round from mobile-number OTP (which had no real SMS provider behind it; it only ever simulated delivery) to real email delivery via Resend. A single flow handles both signup and login: if the email isn't recognized, the customer is asked for a name and an account is created on OTP verification. New signups — and any existing account that doesn't have a mobile number on file yet — are then required to enter and verify an Indian mobile number before a full login session is issued; the account row exists in the database at that point, but no session cookie is granted until this step completes.

Field	Detail
Login route	<code>/customer/login</code> → <code>/customer/verify-otp</code> → <code>/customer/add-mobile</code> (only shown if no mobile is on file yet)
Session	JWT in httpOnly cookie <code>customerToken</code> , issued only once both email and mobile are on file
Mobile format	Indian mobile numbers only — <code>^[6-9]\d{9}\$</code> — enforced client-side, server-side, and at the database schema level (with a unique index)
Mobile-pending token	A separate, short-lived (15 minute) JWT scoped only to the "add mobile" step — cannot be used to access any customer data, place orders, or otherwise act as a login session
OTP delivery	Real email via Resend, branded template — no dev bypass code exists anymore (see Section 7)

Admin

Email + password. Two roles: `Admin` and `SuperAdmin` (currently identical permissions — the distinction exists in the schema for future use).

Field	Detail
Login route	<code>/admin/login</code>
Session	JWT in httpOnly cookie <code>adminToken</code>
Seeded account	<code>7alpsadmin@7alps.com</code> / <code>7alps123</code> (run <code>npm run seed</code> in <code>backend/</code> ... see note below)
Forgot password FIXED	OTP email flow — <code>POST /api/v1/auth/forgotPassword</code> → <code>verifyOTP</code> → <code>resetPasswordAfterOTP</code> , delivered via Resend. Verified genuinely end to end this round: the final reset-password step was previously a disconnected form component that silently reloaded the page instead of submitting (no validation, backend never called); it's now properly wired, and the backend also re-checks the OTP at that final step instead of only checking that the reset window hadn't expired.

Careful with `npm run seed`. That script reseeds the *product catalog* from scratch (it deletes existing products first) — it is not the admin-account seeder. The admin account is seeded via `node dev-data/seedAdmin.js` directly, which is safe to re-run (it checks before creating).

B2B (marketing team)

Email + password, same shape as Admin login, but a completely separate account collection. B2B accounts are **created by an admin** — there is no self-signup. Each B2B login represents an internal marketing-team member who sells in bulk on behalf of a business buyer.

Field	Detail
Login route	<code>/b2b/login</code>
Session	JWT in httpOnly cookie <code>b2bToken</code>
Account creation	Admin → B2B Team → Add member
Seeded account	<code>b2bpartner@7alps.com</code> / <code>7alps123</code> (<code>npm run seed:b2b</code>)
Forgot password FIXED	Same OTP flow as Admin — <code>POST /api/v1/b2b/forgotPassword</code> → <code>verifyOTP</code> → <code>resetPasswordAfterOTP</code> , delivered via Resend. Two bugs fixed this round: the frontend was calling this endpoint without its <code>/b2b</code> URL prefix, so every request 404'd before reaching the backend at all; and the same broken final reset-password step described in the Admin row above. Both are now fixed and verified end to end from the actual login page.

3. Route Map

Public (no login)

Route	Page
/	Home — hero, categories, featured products, process, testimonials, FAQs
/products	Full catalog with filters (category, price, search, rating sort)
/products/:id	Product detail — gallery, info, ingredients, reviews
/our-process	Sourcing/production story
/why-7alps	Brand positioning / comparison
/global-trade	Export/trade information
/b2b-portal	Wholesale marketing/landing page (static content, not the logged-in B2B portal). Its own Contact form now actually saves submissions FIXED (same backend as the main Contact page), and every CTA on the page (tier cards, order builder) scrolls to it FIXED instead of doing nothing.
/contact	Contact / quote-request form NOW SAVED + map embed
/cart	Shopping cart (guest or logged-in); real (not hardcoded) delivery estimate FIXED ; newsletter signup now actually subscribes FIXED

Customer (requires customer login)

Route	Page
/checkout	Shipping address (with map picker, now auto-filled from the customer's default saved address FIXED) + payment method + place order. Also handles standalone "Buy Now" purchases FIXED . Blocks the order with a warning banner if the delivery state isn't on the admin's serviceable-states list NEW — see Section 8.
/customer/orders	Order history — product photo + expected delivery date per item NEW , "Order Again" NEW , "Rate this product" on delivered items NEW , search by ID/product/status/address IMPROVED
/customer/profile	Personal details + address book (with map picker); saving your only address (or none marked default) now auto-sets it as default FIXED
/customer/wishlist	Saved products via the heart icon on any product card NEW

Admin (requires admin login)

Route	Page
<code>/admin</code>	Dashboard — customers, orders, products, revenue, plus a Delivery Settings card and a Serviceable States card NEW (Section 8)
<code>/admin/products</code>	Product CRUD, variants, image upload, admin-set display rating NEW
<code>/admin/categories</code>	Category CRUD
<code>/admin/coupons</code>	Coupon CRUD: percentage or flat discounts, minimum order amount, one-time or reusable per customer, activate/deactivate NEW (Section 10)
<code>/admin/orders</code>	All orders (customer + B2B); now server-side paginated and searchable by ID/customer/product/address/payment REBUILT , plus expected delivery date per order NEW
<code>/admin/customers</code>	Customer list with order/spend stats
<code>/admin/reviews</code>	Review moderation (genuine customer reviews, prompted after delivery)
<code>/admin/b2b</code>	B2B Team — members with sortable, month-filterable earnings NEW ; quote approval with working search FIXED
<code>/admin/contacts</code>	Contact form submissions — view, mark New/Read/Resolved, delete NEW
<code>/admin/support-tickets</code>	B2B "Contact Support" submissions NEW — server-side paginated and searchable (subject/message/member), status workflow, delete. See Section 4.

B2B (requires B2B login)

Route	Page
<code>/b2b</code>	Dashboard — quotes, orders, sales stats. The "Contact Support" button in the sidebar now actually saves a ticket for admin FIXED instead of just faking a "sent" state locally.
<code>/b2b/catalog</code>	Browse products, propose bulk price/qty
<code>/b2b/request-quote</code>	Buyer details (with map picker) + submit + quote history, search now working FIXED
<code>/b2b/orders</code>	Orders created from approved quotes, searchable by order ID or product

4. Feature Walkthrough

Customer-facing storefront

- **Browse & filter** — category, price, search, rating sort (now backed by real, admin-set product ratings — see Section 5).

- **Wishlist** **NEW** — heart icon on any product card (listing pages, product detail, related products); toggles instantly, redirects a guest to login. Dedicated `/customer/wishlist` page lists everything saved.
- **Cart** — guest cart in localStorage; merges into the server cart automatically on login. Decreasing quantity below 1 now removes the item instead of silently doing nothing **FIXED**.
- **Buy Now** **FIXED** — purchasing directly from a product page no longer touches the cart at all; checkout shows only that one item, and the customer's actual cart is left completely untouched.
- **Checkout** — shipping address form with an embedded Google Maps picker, auto-filled from the customer's default saved address; choice of **Cash on Delivery** or **Pay Online (Razorpay)** — COD only shown at all if the admin has it enabled **NEW** (Section 9). If the resolved delivery state isn't on the admin's serviceable-states allow-list, a red banner explains the order can't be placed and the Place Order/Pay button disables **NEW** — see Section 8.
- **Coupons** **NEW** — a coupon field at checkout plus an "Available offers" list of active codes; the discount is previewed live, re-checked at order time, and shown on the order (Section 10).
- **Orders** — history with status, itemized breakdown (product photo + expected delivery date per item), shipping address, broadened search (ID/product/status/address/date), and an "Order Again" button that re-adds every item from a past order to the cart **NEW**.
- **Profile** — editable name only (email is the verified account identity set at signup, and is no longer editable from the profile page **CHANGED**; mobile was already locked); a real address book (add/edit/delete/set-default), each entry fillable via the same map picker. The only (or first) saved address is now always guaranteed to be the default **FIXED**.
- **Reviews** — logged-in customers can rate & review a product from the product page, and are now also prompted with a "Rate this product" link on delivered order items **NEW** (skipped once already reviewed); reviews are held as **Pending** until an admin publishes them.

Admin console

- **Dashboard** — live counts: customers, orders, products, revenue; plus a **Delivery Settings** card, a **Cash on Delivery** toggle **NEW** (Section 9), and a **Serviceable States** card **NEW** to restrict which Indian states/UTs the store accepts orders from — see Section 8.
- **Products** — full CRUD, multiple variants per product (label/price/MRP/stock), multi-image upload to Cloudflare R2, plus an admin-set display rating (average + count) **NEW** — see Section 5.
- **Categories** — add/edit/activate/delete; delete is blocked if products still reference the category, with a clear error explaining why.
- **Coupons** **NEW** — create percentage or flat-amount codes with a minimum order value and a one-time or reusable usage rule; activate, deactivate, or delete unused ones (Section 10).
- **Orders** **REBUILT** — every order (storefront and B2B) in one place; now paginated and searched server-side (order ID, customer, product, address, payment, status) instead of fetching the entire order collection on every load; inline status changes (Confirmed → Processing → Shipped → Delivered/Cancelled); expected delivery date per order.
- **Customers** — every registered customer with their order count and lifetime spend.
- **Reviews** — moderate: publish, flag, or delete any submitted review.
- **Contact Submissions** **NEW** — every Contact Us / quote-request form submission, with New/Read/Resolved status and search.
- **Support Tickets** **NEW** — every message a B2B member sends via the "Contact Support" button on their dashboard (previously not saved anywhere at all — the form faked a success message and discarded the

input). Server-side paginated and searchable by subject, message, or the submitting member's name/email/business, with the same New/Read/Resolved workflow as Contact Submissions.

- **B2B Team** — create marketing-team logins; each member's bulk orders and earnings, now sortable and filterable by month **NEW** ("earnings" = value of orders created once their quotes are approved — see the Earnings note below); review a pending quote (adjust quantity/price if needed), approve it — which atomically decrements stock and creates a real order — or reject it with a reason. Quote search now actually works **FIXED** (was previously silently broken — it searched a field that was never populated).

B2B wholesale portal

- **Catalog** — the same live product catalog; the member picks a variant, a bulk quantity (10+ units), and proposes their own discounted unit price (capped at the listed retail price).
- **Request Quote** — the products picked in Catalog collect into a draft; the member adds the buyer's business name and delivery address (map picker included) and submits it for admin approval. Past quotes and their status are listed below the form, with working search **FIXED**.
- **Orders** — once admin approves a quote, it becomes a real order here; searchable by order ID or product name, alongside the existing status filter.
- **Dashboard** — quote counts by status, total orders, total sales. Its "Contact Support" button/modal now actually submits to the admin's new Support Tickets page **FIXED** instead of silently discarding the message.

5. Product Ratings **NEW**

Every product now has a display rating — an average (0–5 stars) and a count of "people rated" — shown on product cards and the product detail page, and usable for "Sort by Rating" on the catalog.

Admin-entered, not yet computed from real reviews. Since the storefront is only just launching, these numbers are set directly by the admin on the product form (`/admin/products`) rather than calculated from the Review collection. A product with no rating set shows a "New" badge instead of stars/zero. Genuine customers can still rate & review products (prompted after delivery, moderated by admin at `/admin/reviews`) — that data is collected in parallel and is intended to eventually replace the admin-set number once there's enough real review volume, but that switch-over has not been built yet.

Two related bugs were fixed as part of this: the product detail page's star rating used to be hardcoded to 5 stars for every product regardless of its actual rating, and the catalog's "Sort by Rating" was a silent no-op because the backend had neither a rating field to sort by nor a query-param-aware sort at all.

6. Payments

Cash on Delivery is on by default and can be switched off entirely by the admin **NEW** (Section 9), leaving Pay Online as the only option. **Razorpay is fully live and has been verified with a real completed test transaction** — order creation, the checkout modal, signature verification, and order confirmation all work end to end. If `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` in `backend/.env` are ever blanked out, attempting to pay online returns a clear "not configured yet" error rather than failing silently — it never crashes the server either way.

How it works: the backend computes the charge amount itself — from the customer's live server-side cart for a normal checkout, or from the specific item(s) requested for a Buy Now purchase — it never trusts a client-supplied amount, which is the standard payment-security practice (a client could otherwise tamper with what it pays). `POST /customer/payments/razorpay-order` creates the Razorpay order sized to that amount; the frontend opens Razorpay's Standard Checkout modal with it; on success, `POST /customer/payments/razorpay-verify` checks the HMAC-SHA256 signature Razorpay returns before creating the real order, decrementing stock, and (for a cart checkout) clearing the cart.

7. Email (Resend)

All transactional email — Customer email-OTP login **NEW**, Admin signup OTP, both the Admin and B2B forgot-password OTP flows, and the contact-form admin notification — sends through Resend instead of the previous Gmail/Nodemailer setup.

Branded HTML template **NEW**. Every one of the emails above previously sent as bare, unstyled plain text (no logo, no color, no layout at all). All six now render through one shared HTML template — `backend/utils/emailTemplates.js` — using the site's actual brand colors (`#0F6B3E` primary green, `#16442C` header, cream background) and the 7ALP's logo, with a plain-text version still sent alongside for email clients that prefer it. `backend/utils/email.js`'s `sendEmail()` signature grew one optional field (`html`) — existing callers that don't pass it are unaffected.

Field	Detail
Config	<code>RESEND_API_KEY</code> + <code>RESEND_FROM_EMAIL</code> in <code>backend/.env</code>
Sender address	<code>7ALps <support@7alpsglobal.com></code> — <code>7alpsglobal.com</code> is verified with Resend, so email delivers to any recipient (earlier in setup, this pointed at a malformed address and OTP emails failed to send outright; fixed previously)
Contact form notification	Optional — if <code>CONTACT_NOTIFY_EMAIL</code> is set in <code>backend/.env</code> , admin gets an email on every new Contact Us submission; unset by default so it's silently skipped until a real address is configured.

Newsletter signup **NEW**

The "Stay Connected" banner on the Cart page previously had no working input or button at all. It now actually captures an email via `POST /api/v1/newsletter/subscribe` (public, idempotent — resubscribing the same address just succeeds again) into a dedicated `NewsletterSubscribers` collection.

Deliberately minimal. There's no admin UI page for this yet (only `GET /api/v1/admin/newsletter` to list subscribers, callable but not wired to any screen), no unsubscribe flow, and no actual campaign-sending capability — this is capture-only for now, matching the "minimal" scope this was built to.

Domain verified — sandbox restriction lifted. With `7alpsglobal.com` verified, OTP and notification emails send to any recipient, not just the Resend account owner's own inbox (Resend's default sandbox behavior for unverified senders).

8. Serviceable States (Delivery Area Restriction) NEW

The admin can restrict which Indian states/UTs the store accepts orders from. When the restriction is off (the default), orders are accepted from anywhere, exactly as before this feature existed.

Admin configuration

On the `/admin` dashboard, the **Serviceable States** card lets the admin toggle "Restrict orders to selected states" and check off any combination of the 28 states + 8 union territories. Turning the toggle on is blocked (both client- and server-side) until at least one state is checked, so the admin can't accidentally lock out every customer with an empty allow-list. A checked box means "we ship here" — unchecked blocks orders from that state. The card also shows Check-all/Clear-all shortcuts, tints checked rows, and a live "Shipping to X of Y — Z blocked" summary, added after an early test accidentally left nearly every state checked because the original copy read ambiguously.

Customer enforcement

The restriction is enforced at two layers:

- **Checkout UI** — as soon as the shipping state resolves (typed manually, or auto-filled by the Google Maps picker), the checkout page checks it against the admin's list (fetched from the public settings endpoint). If it isn't allowed, a red banner reads "Sorry, we don't currently accept orders from <state>" and lists the states currently served; the Place Order / Pay button disables.
- **Backend (authoritative)** — the same check runs again on the server for every order-placement path, so the restriction can't be bypassed by calling the API directly: `POST /customer/orders` (COD), `POST /customer/payments/razorpay-order` (checked *before* the Razorpay order — and therefore before any charge — is created), and `POST /customer/payments/razorpay-verify` (re-checked as a guard in case the admin's list changes in the few seconds between the two Razorpay steps). A blocked request returns `422` with the same "not currently accepted" message.

Matching is on the state name customers already type. The shipping address form has always had a free-text State field (auto-filled from Google's `administrative_area_level_1` when the map picker is used). The serviceability check compares that value to the admin's list case-insensitively and trimmed, so "karnataka " and "Karnataka" both match — no new dropdown or pincode lookup was introduced.

Field	Detail
Data model	<code>Settings.serviceableStatesEnabled</code> (Boolean, default <code>false</code>) + <code>Settings.serviceableStates</code> (String array, default <code>[]</code>) — same singleton document as Delivery Settings
Admin API	<code>GET/PATCH /api/v1/admin/settings</code> — the existing endpoint, extended to also accept <code>serviceableStatesEnabled</code> / <code>serviceableStates</code> alongside <code>expectedDeliveryDays</code> , each independently optional
Public API	<code>GET /api/v1/settings/public</code> now also returns <code>serviceableStatesEnabled</code> / <code>serviceableStates</code> , alongside the existing <code>expectedDeliveryDays</code>
Shared logic	<code>backend/utils/serviceability.js</code> — one <code>assertStateServiceable()</code> helper used identically by the COD and both Razorpay steps, so all three paths can never drift out of sync

9. Cash on Delivery Toggle NEW

The admin can turn Cash on Delivery off entirely, leaving Pay Online (Razorpay) as the only checkout option — for example while COD fulfillment isn't ready in a given phase of the rollout. Default is on, matching all prior behavior.

1 Toggle it

The **Cash on Delivery** card on the `/admin` dashboard (next to Delivery Settings) has a single "Accept Cash on Delivery" switch. Saving calls the same `PATCH /api/v1/admin/settings` endpoint used by Delivery Settings and Serviceable States, extended with one more independent field, `codEnabled`.

2 Checkout reacts immediately

When off, the Cash on Delivery radio option is hidden entirely at `/checkout` — a customer only ever sees Pay Online. If a customer had COD selected before the setting changed mid-session, the page silently switches their selection to Pay Online rather than leaving a hidden, stale choice active.

3 Server-side, not just hidden in the UI

`POST /customer/orders` — the only endpoint that ever creates a COD order (Razorpay orders are created and verified through its own separate routes) — checks `Settings.codEnabled` and returns `400` with "Cash on Delivery is currently unavailable. Please pay online instead." if it's off, so the restriction can't be bypassed by calling the API directly even while the toggle is off.

Field	Detail
Data model	<code>Settings.codEnabled</code> (Boolean, default <code>true</code>) — same singleton document as Delivery Settings and Serviceable States
Public API	<code>GET /api/v1/settings/public</code> now also returns <code>codEnabled</code>
Enforcement point	<code>orderController.createOrder</code> only — Razorpay's own two-step flow (Section 8's table) is unaffected by this toggle, since it's a separate payment method entirely

10. Coupons NEW

Admin-managed discount codes, set up at `/admin/coupons`. Customers apply them on the checkout page, the discount appears in the order summary, and it's stored on the placed order.

1 Create the coupon (admin)

A code, an optional description, a discount type (**percentage**, 1–100, or **flat** rupee amount), a minimum order amount before shipping (0 means any order), and a usage rule (**reusable**, or **one-time per customer**). A new coupon is active on save; the switch on its card turns it on or off.

2 Customer sees the offers (checkout)

Checkout lists every active coupon as an "Available offers" chip. An offer the customer can't use right now is greyed out: either the order is below its minimum (the chip says how much more is needed), or it's a one-time coupon the customer has already used.

3 Apply a code

The customer types a code or taps an offer. `POST /api/v1/customer/coupons/preview` validates it against the current cart (or the Buy Now item) and returns the discount, shown as a green Discount line. If the cart changes so the coupon no longer qualifies, it's removed automatically and the reason is shown.

4 Order placed

The coupon is validated again on the server, inside the order transaction, for COD (`POST /customer/orders`) and for Razorpay (at order creation and again at verification). The order stores a snapshot of the coupon and the discount amount, and `Coupon.usedCount` increases.

Rule	Detail
Data model	<code>Coupon</code> collection (<code>code</code> unique, <code>discountType</code> , <code>discountValue</code> , <code>minOrderAmount</code> , <code>usageType</code> , <code>active</code> , <code>usedCount</code>). Orders gain <code>coupon</code> (a snapshot) and <code>discountAmount</code> .
Discount maths	Percentage discounts are rounded to whole rupees. Flat discounts are capped at the items total. Shipping is never discounted.
Free shipping	The ₹999 threshold is judged on the items total before the discount, so a coupon never removes free delivery.
One-time rule	One non-cancelled order per customer per coupon code. Cancelling an order frees the coupon for that customer.
Deleting	Allowed only for a coupon that has never been used. Otherwise deactivate it, which keeps the usage record.
Where it's enforced	<code>utils/coupon.js</code> (validation rules) and <code>orderController.priceOrder</code> (pricing), shared by all three order endpoints.
Sample data	<code>npm run seed:coupons</code> adds WELCOME10, FLAT100, SAVE20 and FLAT50. It only inserts codes that don't exist yet, so re-running it changes nothing.

11. Testing Reference

What	Value
Admin login	<code>7alpsadmin@7alps.com</code> / <code>7alps123</code>
B2B login	<code>b2bpartner@7alps.com</code> / <code>7alps123</code>
Customer login CHANGED	Any real email address — OTP is genuinely emailed via Resend (check the inbox; no dev bypass code exists). A new account must then also enter a valid Indian mobile number (e.g. <code>9876543210</code> — must match <code>^[6-9]\d{9}\$</code>) before login completes.
Razorpay test card (recommended)	<code>5267 3181 8797 5449</code> (Mastercard) — any future expiry, any 3-digit CVV
Razorpay test OTP	Any 4-10 digit number (e.g. <code>1234</code>) — it's a test-mode simulation, not a fixed code
Sample coupons (after <code>npm run seed:coupons</code>)	<code>WELCOME10</code> (10% off, min ₹499, one-time) · <code>FLAT100</code> (₹100 off, min ₹999, one-time) · <code>SAVE20</code> (20% off, min ₹1499, reusable) · <code>FLAT50</code> (₹50 off, min ₹499, reusable)
Password-reset OTP (Admin/B2B)	Emailed via Resend from <code>support@7alpsglobal.com</code> , delivered to the real account email — now testable end to end from the actual login page (Section 2), including the final "set new password" step

Avoid the generic 4111 1111 1111 1111 Visa test card — Razorpay's test environment treats it as a foreign-issued card, and most test accounts (this one included) have international card acceptance disabled by default, so it fails with `international_transaction_not_allowed`. The Mastercard number above is domestic and works reliably.

12. Known Limitations

- **Coupon changes during a Razorpay payment** — a coupon is checked when the Razorpay order is created (that sets the amount charged) and again when the payment is verified. If an admin deactivates or changes the coupon while the Razorpay window is open, verification can refuse the order after the payment has been captured. The money is recorded in Razorpay, but the order would need to be created or refunded by hand. Trusting the price recorded at order creation would close this gap; that is a design choice still to be made.
- **One-time coupons aren't strictly atomic** — the once-per-customer check runs inside the order transaction, but two checkouts submitted at the same instant with the same one-time code could both pass it. Unlikely in practice for a single customer; a unique index would close it fully if it ever matters.
- **Serviceable-states matching is exact-name, not geographic** — a customer's typed state is compared against the admin's allow-list by string (trimmed, case-insensitive); it isn't validated against the city/pincode also on the form, so a customer could still type a served state name with a mismatched pincode. Acceptable for the current scope (a state-level, not address-level, restriction).
- **Admin Quotes tab date-range filtering isn't exposed in the UI** — `GET /api/v1/admin/quotes` accepts `dateFrom / dateTo` alongside `status`, but the B2B Team page's Quotes tab UI only exposes a status dropdown (search now works correctly as of this round — that part of the earlier limitation is resolved).
- **Mobile number is validated, not OTP-verified** — the Indian mobile number collected right after email verification is format- and uniqueness-checked, but (unlike email) no confirmation code is sent to it. This is a deliberate scope choice for this round, not an oversight; adding a second, mobile-based OTP step is a possible future enhancement if it's ever needed.
- **The home page's decorative "Shop by category" tiles are static content** (curated imagery in a fixed 3-tile layout), separate from the real, admin-managed Category system used everywhere else (product forms, filters, B2B catalog).
- **No order/quote-status confirmation emails** — order placement, quote approval/rejection, etc. remain visible in-app only; Resend is currently used only for OTP flows and the optional contact-form admin notification.
- **Product ratings are admin-entered, not computed from real reviews** — a deliberate interim state for launch (see Section 5), not a bug; genuine review collection is running in parallel for a future switch-over.
- **Newsletter subscription is capture-only** — no admin page to view/export subscribers (the data and a listing endpoint exist, just no screen for it yet), no unsubscribe flow, no campaign sending.
- **B2B member deletion has no admin UI** — the backend supports actually deleting a B2B member (`DELETE /api/v1/admin/b2b/:id`), but the B2B Team admin page only offers toggling a member active/inactive. Worth confirming whether soft-delete-only is the intended policy.
- **Three unused scaffold folders** — `frontend/src/features/process`, `features/partnerships`, and `features/whyChooseUs` are earlier/duplicate versions of the Our Process and Why 7ALP's pages, superseded by `ourProcess` / `why7ALPs` and not wired to any route. Harmless dead code, not a live "Partnerships" page — the Route Map above has been corrected to drop the stale `/partners` entry a prior pass of this document listed in error.

Full audit pass (2026-08-05). A systematic sweep checked every search/filter config, interactive element, and hardcoded value across the app for the same "looks wired up but isn't" pattern that produced several of the fixes above. Beyond what's listed as fixed in this document, a handful of harmless items were found and intentionally left alone (not asked for, not affecting any live feature): a leftover debug `console.log` on the homepage, a few unused Redux cart reducers superseded by the async thunks actually in use, two components with dead buttons that aren't rendered anywhere in the app, and some stale commented-out code in `userRoutes.js` / `Footer.jsx`. None of these are live bugs.

Backend error-handling & auth hardening (2026-08-06). A second pass, triggered by a raw MongoDB error message (`E11000 duplicate key...`) leaking straight to the admin UI on a duplicate B2B email. Root cause: the global error handler only produced friendly messages in production — in development it returned the raw driver error unmodified. Fixed app-wide, so every duplicate-key and validation error now returns a clear, human-readable message (e.g. *"This email address is already in use"*) regardless of environment, not just for B2B. While auditing the rest of the error-handling and auth code, two more real issues turned up and were fixed: the Admin/B2B "reset password after OTP" endpoint accepted the request without re-checking the OTP (only the expiry window), and request logging (`morgan`) was silently disabled outside development, so a production-run server produced zero access logs.

Customer auth rework & branded email templates (2026-08-09). Customer login/signup moved from mobile-number OTP (which had never had a real SMS provider behind it — it only simulated delivery and accepted a universal `123456` bypass code) to real email OTP delivery via Resend, per a direct request to drop SMS OTP entirely. A new mandatory step now collects and validates an Indian mobile number right after email verification, before a full login session is granted — see Section 2. Separately, every transactional email the platform sends (not just the new customer flow) was rebuilt from plain unstyled text onto one shared branded HTML template — see Section 7.

Serviceable states / delivery-area restriction (2026-09-13). Added at the admin's request to limit which Indian states the store ships to while the business scales up gradually. Enforced identically at checkout (client-side warning) and on every order-placement API path (server-side, authoritative) — see Section 8.

Cash on Delivery toggle (2026-09-13). A same-day follow-up request: the admin can now switch COD off entirely, leaving Pay Online as the only checkout option, enforced both by hiding the option in the UI and by rejecting `POST /customer/orders` server-side when it's off — see Section 9.

Coupons (2026-10-06). Admin-managed discount codes, percentage or flat, with a minimum order value and a one-time or reusable usage rule. Customers apply them at checkout, where they're previewed live and re-checked on the server when the order is created. The discount is stored on each order. See Section 10.